

Physical ER Plan — AITasker v2 (PostgreSQL)

Fully updated to reflect the v5 logical ER. 51 tables total.

Global Conventions

- **All primary keys:** `UUID NOT NULL DEFAULT gen_random_uuid()`
- **All timestamps:** `TIMESTAMPZ` (UTC)
- **All money:** `BIGINT` (VND as integer đồng — no decimals)
- **All JSON:** `JSONB`
- **Enums:** `TEXT` with `CHECK` constraints (not PostgreSQL `ENUM` — easier migrations)
- **Soft nullability:** every column `NOT NULL` unless explicitly marked `NULL`
- **Extension required:** `CREATE EXTENSION IF NOT EXISTS "pgcrypto";`

Section 1 — Users & Role Profiles

Table: `users`

Column	Type	Constraints	Notes
<code>id</code>	<code>UUID</code>	<code>PK NOT NULL DEFAULT gen_random_uuid()</code>	
<code>email</code>	<code>TEXT</code>	<code>NOT NULL UNIQUE</code>	
<code>password_hash</code>	<code>TEXT</code>	<code>NOT NULL</code>	bcrypt
<code>full_name</code>	<code>TEXT</code>	<code>NOT NULL</code>	
<code>phone</code>	<code>TEXT</code>	<code>NULL</code>	
<code>roles</code>	<code>JSONB</code>	<code>NOT NULL DEFAULT '[]'</code>	e.g. [<code>"CLIENT_CEO"</code> , <code>"EXPERT"</code>]
<code>active_role</code>	<code>TEXT</code>	<code>NOT NULL CHECK (active_role IN ('CLIENT','EXPERT','ADMIN'))</code>	JWT claim evaluated by all guards
<code>client_subtype</code>	<code>TEXT</code>	<code>NULL CHECK (client_subtype IN ('CEO','TECH_TEAM'))</code>	NULL for EXPERT and ADMIN
<code>subscription_client_tier</code>	<code>TEXT</code>	<code>NOT NULL DEFAULT 'free' CHECK (subscription_client_tier IN ('free','pro'))</code>	
<code>subscription_expert_tier</code>	<code>TEXT</code>	<code>NOT NULL DEFAULT 'free' CHECK (subscription_expert_tier IN ('free','pro'))</code>	
<code>sub_client_expires_at</code>	<code>TIMESTAMPZ</code>	<code>NULL</code>	NULL when tier = free
<code>sub_expert_expires_at</code>	<code>TIMESTAMPZ</code>	<code>NULL</code>	NULL when tier = free
<code>sepay_bank_account_xid</code>	<code>TEXT</code>	<code>NULL</code>	Set via Bank Hub Hosted Link webhook
<code>bank_account_holder_name</code>	<code>TEXT</code>	<code>NULL</code>	
<code>bank_linked_at</code>	<code>TIMESTAMPZ</code>	<code>NULL</code>	
<code>self_technical</code>	<code>BOOLEAN</code>	<code>NOT NULL DEFAULT FALSE</code>	Scenario B flag
<code>is_active</code>	<code>BOOLEAN</code>	<code>NOT NULL DEFAULT TRUE</code>	
<code>created_at</code>	<code>TIMESTAMPZ</code>	<code>NOT NULL DEFAULT now()</code>	

Indexes: `email` (unique), `active_role`, `client_subtype`

Table: `client_profiles`

Column	Type	Constraints
<code>user_id</code>	<code>UUID</code>	<code>PK NOT NULL REFERENCES users(id) ON DELETE CASCADE</code>
<code>company_name</code>	<code>TEXT</code>	<code>NULL</code>
<code>industry</code>	<code>TEXT</code>	<code>NULL</code>
<code>ceo_name</code>	<code>TEXT</code>	<code>NULL</code>

Table: `expert_profiles`

Column	Type	Constraints	Notes
user_id	UUID	PK NOT NULL REFERENCES users(id) ON DELETE CASCADE	
bio	TEXT	NULL	
engagement_model	TEXT	NULL CHECK (engagement_model IN ('MILESTONE', 'HOURLY', 'HYBRID'))	
archetype_history_json	JSONB	NULL	NEW — self-declared archetype history. Format: <code>[{archetype_code, tier, self_declared: true}]</code> . Seeds the 20% Archetype-tier congruence component in composite matching score for cold-start experts with no closed platform engagements. Confirmed history derived at query time from <code>engagements</code> → <code>projects</code> ; not stored here.

Table: tech_team_profiles

Column	Type	Constraints	Notes
user_id	UUID	PK NOT NULL REFERENCES users(id) ON DELETE CASCADE	
linked_client_id	UUID	NOT NULL REFERENCES users(id)	CEO user who issued the handoff link
linked_project_id	UUID	NOT NULL REFERENCES projects(id)	Scope guard: account locked to one project
role_title	TEXT	NULL	e.g. "CTO", "Lead Backend"

Index: linked_project_id

Section 2 — Expert Capability Sub-entities

Table: expert_domain_depths

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id) ON DELETE CASCADE	
domain_code	TEXT	NOT NULL	'DOMAIN_A' through 'DOMAIN_F'
depth_level	TEXT	NOT NULL CHECK (depth_level IN ('SURFACE', 'OPERATIONAL', 'DEEP'))	
verification_tier	TEXT	NOT NULL DEFAULT 'CLAIMED' CHECK (verification_tier IN ('CLAIMED', 'EVIDENCE_BACKED', 'SCENARIO_VERIFIED', 'PLATFORM_DEMONSTRATED'))	

Unique constraint: (expert_id, domain_code)

Index: expert_id

Table: expert_seam_claims

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id) ON DELETE CASCADE	
seam_code	TEXT	NOT NULL	e.g. 'SEAM_AC', 'SEAM_DE'
verification_tier	TEXT	NOT NULL DEFAULT 'CLAIMED' CHECK (verification_tier IN ('CLAIMED', 'EVIDENCE_BACKED', 'SCENARIO_VERIFIED', 'PLATFORM_DEMONSTRATED'))	
submission_count	INT	NOT NULL DEFAULT 0	5-attempt throttle
locked_until	TIMESTAMPZ	NULL	30-day lockout after 5th failure

Unique constraint: (expert_id, seam_code)

Index: expert_id, seam_code

Table: expert_stack_tags

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id) ON DELETE CASCADE
tag	TEXT	NOT NULL

Unique constraint: (expert_id, tag)

Index: expert_id

Table: portfolio_submissions

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id) ON DELETE CASCADE	
seam_claim_id	UUID	NOT NULL REFERENCES expert_seam_claims(id)	Targets which seam claim is being upgraded to Tier 2
project_description	TEXT	NOT NULL	
decision_points	TEXT	NOT NULL	Structured rubric response
status	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED'))	
llm_confidence	FLOAT	NULL	Set after LLM evaluation; threshold ≥ 0.85 for auto-upgrade
submitted_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
evaluated_at	TIMESTAMPZ	NULL	

Index: expert_id, seam_claim_id

Table: scenario_assessments

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
seam_code	TEXT	NOT NULL	
question_text	TEXT	NOT NULL	
rubric_json	JSONB	NOT NULL	Each dimension must pass independently
last_used_at	TIMESTAMPZ	NULL	System selects lowest to prevent gaming

Index: seam_code, last_used_at

Table: scenario_responses

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
assessment_id	UUID	NOT NULL REFERENCES scenario_assessments(id)	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id) ON DELETE CASCADE	
seam_claim_id	UUID	NOT NULL REFERENCES expert_seam_claims(id)	Targets which seam claim is being upgraded to Tier 3
response_text	TEXT	NOT NULL	
pass_fail	TEXT	NULL CHECK (pass_fail IN ('PASS', 'FAIL', 'PENDING'))	Set after LLM rubric evaluation
llm_rubric_scores_json	JSONB	NULL	Per-dimension scores; all must pass
submitted_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Index: expert_id, assessment_id, seam_claim_id

Table: `expert_seam_outcome_signals` (NEW)

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>	
<code>engagement_id</code>	UUID	NOT NULL REFERENCES <code>engagements(id)</code>	
<code>expert_id</code>	UUID	NOT NULL REFERENCES <code>expert_profiles(user_id)</code>	
<code>reviewer_id</code>	UUID	NOT NULL REFERENCES <code>users(id)</code>	TECH_TEAM user who submitted the structured review
<code>outcome_signal_id</code>	UUID	NOT NULL REFERENCES <code>outcome_signals(id)</code>	Aggregate record that generated this row
<code>seam_claim_id</code>	UUID	NOT NULL REFERENCES <code>expert_seam_claims(id)</code>	Direct FK — enforces referential integrity for the Tier 4 UPDATE
<code>seam_code</code>	TEXT	NOT NULL	
<code>signal_type</code>	TEXT	NOT NULL CHECK (<code>signal_type</code> IN ('LOAD_BEARING', 'SIGNIFICANT', 'CONTRIBUTING'))	
<code>valence</code>	TEXT	NOT NULL CHECK (<code>valence</code> IN ('POSITIVE', 'NEGATIVE'))	NEGATIVE feeds the matching hard gate exclusion rule
<code>seam_role</code>	TEXT	NOT NULL	How central this seam was to the actual engagement delivery
<code>created_at</code>	TIMESTAMPTZ	NOT NULL DEFAULT <code>now()</code>	

Index: `expert_id`, `seam_code`, `valence`, `signal_type` — supports both Tier 4 accumulation query and matching hard gate query

Tier 4 accumulation rule evaluated by NestJS on every engagement close:

- 1× `LOAD_BEARING` + `valence=POSITIVE` → immediate Tier 4
- 2× `SIGNIFICANT` + `valence=POSITIVE` (across all engagements) → Tier 4
- 3× `CONTRIBUTING` + `valence=POSITIVE` (across all engagements) → Tier 4

Matching hard gate: 2+ `NEGATIVE` signals on a seam that is load-bearing in the footprint → expert excluded from shortlist for that project

Section 3 — Wallet, Finance & Subscriptions

Table: `wallets`

Column	Type	Constraints
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>
<code>user_id</code>	UUID	NOT NULL UNIQUE REFERENCES <code>users(id)</code> ON DELETE CASCADE
<code>available_balance</code>	BIGINT	NOT NULL DEFAULT 0 CHECK (<code>available_balance</code> >= 0)
<code>locked_balance</code>	BIGINT	NOT NULL DEFAULT 0 CHECK (<code>locked_balance</code> >= 0)

Seeded system wallet at deploy:

```
INSERT INTO users (id, email, password_hash, full_name, roles, active_role, is_active, created_at)
VALUES ('00000000-0000-0000-0000-000000000001', 'platform@aitasker.internal',
       'SEEDNO_LOGIN', 'AITasker Platform', ['ADMIN'], 'ADMIN', true, now());
INSERT INTO wallets (user_id, available_balance, locked_balance)
VALUES ('00000000-0000-0000-0000-000000000001', 0, 0);
```

Store returned wallet `id` as env var `PLATFORM_FEE_WALLET_ID`. All `PLATFORM_FEE` ledger entries credit this row.

Table: `wallet_transactions`

Immutable ledger — rows are **never updated** after insert.

Column	Type	Constraints
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>
<code>wallet_id</code>	UUID	NOT NULL REFERENCES <code>wallets(id)</code>

Column	Type	Constraints
amount	BIGINT	NOT NULL CHECK (amount > 0)
transaction_type	TEXT	NOT NULL CHECK (transaction_type IN ('TOP_UP', 'SUBSCRIPTION', 'ESCROW_LOCK', 'ESCROW_RELEASE', 'PLATFORM_FEE', 'ESCROW_REFUND', 'ESCROW_SPLIT'))
reference_id	TEXT	NULL
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()

Index: wallet_id, transaction_type, created_at

Idempotency constraint — critical for SePay IPN retries:

```
CREATE UNIQUE INDEX wallet_tx_idempotency
ON wallet_transactions(wallet_id, reference_id)
WHERE reference_id IS NOT NULL;
```

Without this, two simultaneous SePay retries can both pass the application-layer check and both INSERT, double-crediting the wallet.

Table: virtual_accounts

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
entity_type	TEXT	NOT NULL CHECK (entity_type IN ('WALLET_TOPUP', 'MILESTONE', 'SERVICE', 'SUBSCRIPTION'))	
entity_id	TEXT	NOT NULL	Polymorphic: user_id for TOPUP · milestone_id for MILESTONE · engagement purchase_reference for SERVICE · user_subscriptions.id for SUBSCRIPTION
va_number	TEXT	NOT NULL UNIQUE	SePay-issued bank account number
fixed_amount	BIGINT	NULL	NULL for WALLET_TOPUP; set for MILESTONE / SERVICE / SUBSCRIPTION
expires_at	TIMESTAMPZ	NULL	24h for MILESTONE VAs; NULL for WALLET_TOPUP
status	TEXT	NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'EXPIRED', 'USED'))	

Index: va_number (unique), entity_id, entity_type, status

SUBSCRIPTION VA entity_id note: Create the user_subscriptions row first with status = 'PENDING_PAYMENT', then use its id as entity_id. On IPN confirmation, UPDATE user_subscriptions.status = 'ACTIVE'.

Table: user_subscriptions

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
user_id	UUID	NOT NULL REFERENCES users(id) ON DELETE CASCADE	

Column	Type	Constraints	Notes
role_type	TEXT	NOT NULL CHECK (role_type IN ('client','expert'))	One row per role per user
tier	TEXT	NOT NULL CHECK (tier IN ('free','pro'))	
status	TEXT	NOT NULL DEFAULT 'PENDING_PAYMENT' CHECK (status IN ('PENDING_PAYMENT', 'ACTIVE', 'EXPIRING_SOON', 'EXPIRED'))	NEW — <code>PENDING_PAYMENT</code> needed so the row exists before IPN fires and can be used as the VA entity_id
price_paid	BIGINT	NOT NULL DEFAULT 0	0 for free tier
activated_at	TIMESTAMPTZ	NULL	Set when IPN confirms payment; NULL for free tier
expires_at	TIMESTAMPTZ	NULL	NULL for free tier

Unique constraint: (user_id, role_type)
Index: user_id, status

Table: withdrawal_requests

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id)	
type	TEXT	NOT NULL DEFAULT 'EXPERT_MANUAL' CHECK (type IN ('MILESTONE_RELEASE', 'EXPERT_MANUAL'))	NEW — <code>MILESTONE_RELEASE</code> auto-created by NestJS on milestone APPROVED; <code>EXPERT_MANUAL</code> by expert-initiated cash-out. Distinguishes the two on the earnings dashboard.
amount	BIGINT	NOT NULL CHECK (amount > 0)	
bank_account_xid	TEXT	NOT NULL	SePay bank account reference
disbursement_id	TEXT	NULL	SePay chi hộ response ID; set after API call
status	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'PROCESSING', 'COMPLETED', 'FAILED'))	
requested_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
confirmed_at	TIMESTAMPTZ	NULL	Set when SePay credit IPN fires
telegram_notified_at	TIMESTAMPTZ	NULL	NEW — Spec §F12: "Withdrawal audit trail: Telegram notification timestamp." Set after bot message confirmed sent for bank_account_xid deduplication fraud alert.

Index: expert_id, status, disbursement_id, bank_account_xid

Section 4 — Organizations

Table: organizations

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
name	TEXT	NOT NULL	
type	TEXT	NULL	
owner_user_id	UUID	NOT NULL REFERENCES users(id)	
subscription_tier	TEXT	NOT NULL DEFAULT 'free'	
payout_designated_user_id	UUID	NULL REFERENCES users(id)	Which member receives org payouts
sepay_bank_account_xid	TEXT	NULL	

Index: owner_user_id

Table: organization_members

Column	Type	Constraints
--------	------	-------------

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
org_id	UUID	NOT NULL REFERENCES organizations(id) ON DELETE CASCADE
user_id	UUID	NOT NULL REFERENCES users(id) ON DELETE CASCADE
org_role	TEXT	NOT NULL CHECK (org_role IN ('OWNER','MEMBER','BILLING'))
platform_roles	JSONB	NOT NULL DEFAULT '[]'
active	BOOLEAN	NOT NULL DEFAULT TRUE

Unique constraint: (org_id, user_id)
Index: org_id, user_id

Section 5 — Elicitation Engine (NEW section)

Table: elicitation_sessions

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
user_id	UUID	NOT NULL REFERENCES users(id)	CEO who started the session
current_stage	INT	NOT NULL DEFAULT 1 CHECK (current_stage BETWEEN 1 AND 5)	Which stage the CEO is currently on
archetype	TEXT	NULL	Locked after Stage 2 CEO confirmation
scenario_type	TEXT	NULL CHECK (scenario_type IN ('STANDARD','SCENARIO_A','SCENARIO_B'))	Determined at Stage 4. A = no tech team; B = self_technical=true
void_list_json	JSONB	NOT NULL DEFAULT '[]'	SDLC gaps detected by LLM in Stage 1. Format: [{void_code, injected: boolean}]
state	TEXT	NOT NULL DEFAULT 'IN_PROGRESS' CHECK (state IN ('IN_PROGRESS','COMPLETED','ABANDONED','RETURNED'))	RETURNED = failed auto-publish gate; CEO re-enters at current_stage
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Updated on every stage advance via trigger or NestJS

Index: user_id, state

Circular FK note: elicitation_sessions does NOT hold project_id. The canonical FK direction is projects.elicitation_session_id → elicitation_sessions(id). Reverse lookup: SELECT * FROM projects WHERE elicitation_session_id = ?.

Section 6 — Projects & Dual Artifacts

Table: projects

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
client_id	UUID	NOT NULL REFERENCES users(id)	CEO user_id
elicitation_session_id	UUID	NULL REFERENCES elicitation_sessions(id)	NEW — NULL for non-elicitation paths. Set by Stage 5 synthesis when creating the project row.
state	TEXT	NOT NULL DEFAULT 'DRAFT' CHECK (state IN ('DRAFT','PUBLISHED','RETURNED_TO_CLIENT','SUSPENDED'))	
archetype	TEXT	NULL	Set by elicitation synthesis engine
tier	TEXT	NULL CHECK (tier IN ('TIER_1','TIER_2','TIER_3'))	
self_technical	BOOLEAN	NOT NULL DEFAULT FALSE	Scenario B flag
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Index: client_id, state, elicitation_session_id

Table: `capability_footprints`

Column	Type	Constraints
<code>project_id</code>	UUID	PK NOT NULL REFERENCES <code>projects(id)</code> ON DELETE CASCADE
<code>required_domains_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>required_seams_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>milestone_framework_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>technical_architecture</code>	TEXT	NULL

Table: `artifact_a`

Column	Type	Constraints	Notes
<code>project_id</code>	UUID	PK NOT NULL REFERENCES <code>projects(id)</code> ON DELETE CASCADE	
<code>state</code>	TEXT	NOT NULL DEFAULT 'DRAFT' CHECK (state IN ('DRAFT', 'PUBLISHED', 'RETURNED_TO_CLIENT', 'SUSPENDED'))	Managed independently from <code>projects.state</code>
<code>business_intent</code>	TEXT	NULL	
<code>physics_constraints_json</code>	JSONB	NULL	Scale, volume, latency constraints
<code>escrow_lock_notice</code>	TEXT	NULL	Displayed to expert: explains Artifact B lock

Table: `artifact_b`

Column	Type	Constraints
<code>project_id</code>	UUID	PK NOT NULL REFERENCES <code>projects(id)</code> ON DELETE CASCADE
<code>tech_stack_json</code>	JSONB	NULL
<code>schema_uploads_json</code>	JSONB	NULL
<code>integration_contracts_json</code>	JSONB	NULL

Access guard (enforced in FastAPI route — not a column):

```
SELECT ab.* FROM artifact_b ab
WHERE ab.project_id = ?
  AND EXISTS (
    SELECT 1 FROM engagements e
    WHERE e.project_id = ab.project_id
    AND e.state >= 'CONNECTED'
    AND e.client_nda_accepted_at IS NOT NULL
    AND e.expert_nda_accepted_at IS NOT NULL
    AND e.expert_id = :current_user_id -- EXPERT path
  )
-- CEO permanently excluded regardless of state – no CEO branch exists
```

Section 7 — Services

Table: `services`

Column	Type	Constraints
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>
<code>expert_id</code>	UUID	NOT NULL REFERENCES <code>expert_profiles(user_id)</code> ON DELETE CASCADE
<code>title</code>	TEXT	NOT NULL
<code>description</code>	TEXT	NOT NULL
<code>domains_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>seams_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>price_vnd</code>	BIGINT	NOT NULL CHECK (price_vnd > 0)

Column	Type	Constraints
state	TEXT	NOT NULL DEFAULT 'DRAFT' CHECK (state IN ('DRAFT','PUBLISHED','SUSPENDED'))
service_type	TEXT	NOT NULL CHECK (service_type IN ('AI_SERVICE','TECH_DISCOVERY'))

Index: expert_id, state, service_type

Section 8 — Engagements

Table: engagements			
Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
project_id	UUID	NULL REFERENCES projects(id)	NULL for SERVICE_PURCHASE / TECH_DISCOVERY
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id)	
service_id	UUID	NULL REFERENCES services(id)	NULL for PROJECT_BASED
type	TEXT	NOT NULL CHECK (type IN ('PROJECT_BASED','SERVICE_PURCHASE','TECH_DISCOVERY'))	Immutable after creation
state	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (state IN ('PENDING','CONNECTED','ACTIVE','CLOSED','DISPUTED'))	SERVICE_PURCHASE is created directly in ACTIVE
connected_at	TIMESTAMPZ	NULL	Set when state → CONNECTED
organization_id	UUID	NULL REFERENCES organizations(id)	
client_nda_accepted_at	TIMESTAMPZ	NULL	NEW — Spec §F5: CEO NDA click-through. Required before CONNECTED transition.
expert_nda_accepted_at	TIMESTAMPZ	NULL	NEW — Spec §F5: Expert NDA acceptance. Required before CONNECTED transition.

Table-level CHECK constraint (type-to-FK enforcement):

```
CONSTRAINT engagement_type_fk_consistency CHECK (  
  (type = 'PROJECT_BASED' AND project_id IS NOT NULL AND service_id IS NULL) OR  
  (type IN ('SERVICE_PURCHASE','TECH_DISCOVERY') AND project_id IS NULL AND service_id IS NOT NULL)  
)
```

Index: project_id, expert_id, state, service_id

Table: spec_clarifications

Surface A — pre-bid. Channel closes when expert submits bid.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
spec_id	UUID	NOT NULL REFERENCES artifact_a(project_id)	
expert_id	UUID	NOT NULL REFERENCES expert_profiles(user_id)	
responded_by	UUID	NULL REFERENCES users(id)	NEW — NULL until ANSWERED. TECH_TEAM on Tier 2+ projects; CEO on Tier 1.
target_component	TEXT	NOT NULL	
question_text	TEXT	NOT NULL	
response_text	TEXT	NULL	

Column	Type	Constraints	Notes
status	TEXT	NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN', 'ANSWERED', 'RESOLVED'))	CHANGED — was 'CLOSED'; spec §0.6 verbatim state machine uses 'RESOLVED'. Any route setting status='CLOSED' fails the CHECK.
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
resolved_at	TIMESTAMPZ	NULL	
Index: spec_id, expert_id			

Section 9 — Non-Linear Bid System

Table: `capability_bids`

Column	Type	Constraints	
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
engagement_id	UUID	NOT NULL UNIQUE REFERENCES engagements(id)	
footprint_alignment_json	JSONB	NULL	
approach_summary	TEXT	NULL	
conditional_pricing_json	JSONB	NULL	
state	TEXT	NOT NULL DEFAULT 'DRAFT' CHECK (state IN ('DRAFT', 'SUBMITTED', 'TECH_REVIEW', 'REVISION_REQUESTED', 'REVISED', 'TECH_APPROVED', 'TECH_DISAPPROVE	
version_number	INT	NOT NULL DEFAULT 1	
Index: engagement_id (unique), state			

Table: `bid_versions`

Immutable snapshot on every REVISED state transition.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
bid_id	UUID	NOT NULL REFERENCES capability_bids(id) ON DELETE CASCADE	
revision_request_id	UUID	NULL REFERENCES bid_revision_requests(id)	NEW — NULL on initial SUBMITTED version; set on all REVISED versions. Links each snapshot to the request that caused it.
version_number	INT	NOT NULL	
snapshot_json	JSONB	NOT NULL	Full bid state at revision time
revised_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
reviser_id	UUID	NOT NULL REFERENCES users(id)	
Unique constraint: (bid_id, version_number)			
Index: bid_id			

Table: `bid_revision_requests`

Surface B — TECH_TEAM flags one component; expert edits only that component.

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
bid_id	UUID	NOT NULL REFERENCES capability_bids(id) ON DELETE CASCADE
flagged_component	TEXT	NOT NULL CHECK (flagged_component IN ('FOOTPRINT','APPROACH','PRICING'))
reason	TEXT	NOT NULL
requested_by	UUID	NOT NULL REFERENCES users(id)
requested_at	TIMESTAMPZ	NOT NULL DEFAULT now()

Index: bid_id

Table: price_negotiations

Surface C — max 2 rounds.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
bid_id	UUID	NOT NULL REFERENCES capability_bids(id) ON DELETE CASCADE	
round_number	INT	NOT NULL CHECK (round_number IN (1,2))	Hard cap
proposer	TEXT	NOT NULL CHECK (proposer IN ('CEO','EXPERT'))	
proposed_pricing_json	JSONB	NOT NULL	
response	TEXT	NULL CHECK (response IN ('ACCEPTED','COUNTERED','DECLINED','PENDING'))	NULL until expert responds
counter_pricing_json	JSONB	NULL	Set when response = COUNTERED
message	TEXT	NULL	
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
expires_at	TIMESTAMPZ	NULL	Round 2 expiry guard

Unique constraint: (bid_id, round_number)

Index: bid_id

Table: bid_conflict_overrides

Surface D — CEO override of TECH_DISAPPROVED bid.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
bid_id	UUID	NOT NULL UNIQUE REFERENCES capability_bids(id)	One override per bid
ceo_user_id	UUID	NOT NULL REFERENCES users(id)	NEW — Spec verbatim: "bid_conflict_overrides row written { reason, risks, overridden_at, ceo_user_id }". Required for Platform Integrity Monitor RQ3 correlation.
override_reason	TEXT	NOT NULL CHECK (char_length(override_reason) >= 50)	50-char minimum enforced at DB level
acknowledged_risks	TEXT	NOT NULL	
overridden_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Section 10 — Milestones (3-Layer System)

Table: milestones

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()

Column	Type	Constraints
engagement_id	UUID	NOT NULL REFERENCES engagements(id) ON DELETE CASCADE
milestone_number	INT	NOT NULL
deliverable_statement	TEXT	NOT NULL
sign_off_authority	TEXT	NOT NULL CHECK (sign_off_authority IN ('TECH_TEAM','CEO','JOINT'))
payment_amount_vnd	BIGINT	NOT NULL CHECK (payment_amount_vnd > 0)
state	TEXT	NOT NULL DEFAULT 'DEFINED' CHECK (state IN ('DEFINED','AWAITING_PAYMENT','FUNDED','IN_PROGRESS','SUBMITTED','IN_REVISION','APPROVED','REL
va_number	TEXT	NULL
va_expires_at	TIMESTAMPZ	NULL
funded_at	TIMESTAMPZ	NULL
submitted_at	TIMESTAMPZ	NULL
approved_at	TIMESTAMPZ	NULL
released_at	TIMESTAMPZ	NULL

review_clock_paused_at TIMESTAMPZ NULL

total_paused_seconds INT NOT NULL DEFAULT 0

Unique constraint: (engagement_id, milestone_number)

Index: engagement_id, state

Table: acceptance_criteria

Layer 1 — payment contract. All is_required = true rows must have verified_at set before APPROVED transition.

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
milestone_id	UUID	NOT NULL REFERENCES milestones(id) ON DELETE CASCADE
criterion_text	TEXT	NOT NULL
is_required	BOOLEAN	NOT NULL DEFAULT TRUE
verified_by_role	TEXT	NOT NULL CHECK (verified_by_role IN ('TECH_TEAM','CEO','JOINT'))
verified_at	TIMESTAMPZ	NULL

Index: milestone_id, is_required

Table: revision_requests

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
criterion_id	UUID	NOT NULL REFERENCES acceptance_criteria(id) ON DELETE CASCADE
requested_by	UUID	NOT NULL REFERENCES users(id)

Column	Type	Constraints
criterion_verdict	TEXT	NULL
feedback_text	TEXT	NOT NULL
requested_at	TIMESTAMP TZ	NOT NULL DEFAULT now()

Index: criterion_id

Table: milestone_dod_items

Layer 2 — expert self-check. Gates the SUBMITTED transition.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
milestone_id	UUID	NOT NULL REFERENCES milestones(id) ON DELETE CASCADE	
item_description	TEXT	NOT NULL	
is_required	BOOLEAN	NOT NULL DEFAULT TRUE	
status	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'COMPLETED', 'NOT_APPLICABLE'))	
completed_at	TIMESTAMP TZ	NULL	
completion_note	TEXT	NULL	Required when status → COMPLETED
not_applicable_note	TEXT	NULL	Required when status → NOT_APPLICABLE
maps_to_criterion_id	UUID	NULL REFERENCES acceptance_criteria(id)	Optional upward link to Layer 1

Table-level CHECK constraint:

```
CONSTRAINT dod_required_cannot_be_na
CHECK (NOT (is_required = TRUE AND status = 'NOT_APPLICABLE'))
```

Spec §0.6 DoD state machine: "NOT_APPLICABLE: only for non-required items." This prevents experts from marking required DoD items as N/A to bypass the SUBMITTED guard.

Index: milestone_id, is_required, status

Table: milestone_sprints

Layer 3 — temporal decomposition. No payment consequence.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
milestone_id	UUID	NOT NULL REFERENCES milestones(id) ON DELETE CASCADE	
sprint_number	INT	NOT NULL	Denormalized label for display
created_by	UUID	NOT NULL REFERENCES users(id)	NEW — Spec §F7 Layer 3 schema verbatim: "created_by FK -- always EXPERT". Route guard enforces active_role = EXPERT.
week_range	TEXT	NOT NULL	e.g. "Week 1-2"
deliverable_checkpoint	TEXT	NOT NULL	
tasks	JSONB	NOT NULL DEFAULT '[]'	[{id, description, status: 'TODO' 'IN_PROGRESS' 'DONE'}]

Unique constraint: (milestone_id, sprint_number)

Index: milestone_id

Table: sprint_status_updates

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>	
<code>milestone_id</code>	UUID	NOT NULL REFERENCES <code>milestones(id)</code> ON DELETE CASCADE	
<code>milestone_sprint_id</code>	UUID	NOT NULL REFERENCES <code>milestone_sprints(id)</code>	NEW — Direct FK replaces implicit <code>(milestone_id, sprint_number)</code> composite lookup. Prevents orphan status updates referencing non-existent sprint plans.
<code>sprint_number</code>	INT	NOT NULL	Denormalized for display only
<code>submitted_by</code>	UUID	NOT NULL REFERENCES <code>users(id)</code>	EXPERT only
<code>submitted_at</code>	TIMESTAMP TZ	NOT NULL DEFAULT <code>now()</code>	
<code>status</code>	TEXT	NOT NULL CHECK (<code>status</code> IN ('ON_TRACK', 'DEVIATION', 'BLOCKER'))	
<code>deviation_note</code>	TEXT	NULL	Required when <code>status</code> = <code>DEVIATION</code>
<code>blocker_type</code>	TEXT	NULL CHECK (<code>blocker_type</code> IN ('SCOPE_GAP', 'TECH_CONSTRAINT', 'EXTERNAL_DEPENDENCY', 'SCOPE_EVOLUTION'))	Required when <code>status</code> = <code>BLOCKER</code>
<code>blocker_description</code>	TEXT	NULL	Required when <code>status</code> = <code>BLOCKER</code>
<code>estimated_resolution</code>	TEXT	NULL	
<code>scope_evolution_flag</code>	BOOLEAN	NOT NULL DEFAULT FALSE	Auto-set when <code>blocker_type</code> = 'SCOPE_EVOLUTION'; triggers <code>addon_phase_requests</code> creation
<code>resolved_at</code>	TIMESTAMP TZ	NULL	
<code>resolved_by</code>	UUID	NULL REFERENCES <code>users(id)</code>	

Index: `milestone_id`, `milestone_sprint_id`, `scope_evolution_flag`

Table: `milestone_submissions`

Column	Type	Constraints
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>
<code>milestone_id</code>	UUID	NOT NULL REFERENCES <code>milestones(id)</code> ON DELETE CASCADE
<code>expert_id</code>	UUID	NOT NULL REFERENCES <code>expert_profiles(user_id)</code>
<code>description</code>	TEXT	NOT NULL
<code>files_json</code>	JSONB	NOT NULL DEFAULT '[]'
<code>submitted_at</code>	TIMESTAMP TZ	NOT NULL DEFAULT <code>now()</code>

Index: `milestone_id`

Table: `paygated_documents`

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>	
<code>milestone_id</code>	UUID	NOT NULL REFERENCES <code>milestones(id)</code> ON DELETE CASCADE	Release trigger: when this milestone → FUNDED via IPN
<code>document_url</code>	TEXT	NOT NULL	

Column	Type	Constraints	Notes
release_state	TEXT	NOT NULL DEFAULT 'STAGED' CHECK (release_state IN ('STAGED', 'RELEASED'))	Flips automatically when IPN fires
staged_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
released_at	TIMESTAMPZ	NULL	Set by IPN handler

Index: milestone_id, release_state

Table: `escrow_accounts`

Structurally changed from original — now supports two mutually exclusive parent paths.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
milestone_id	UUID	NULL REFERENCES milestones(id)	CHANGED to nullable — Path A (PROJECT_BASED): set; Path B: NULL
engagement_id	UUID	NULL REFERENCES engagements(id)	NEW — Path B (SERVICE_PURCHASE / TECH_DISCOVERY): set; Path A: NULL
amount	BIGINT	NOT NULL CHECK (amount > 0)	
client_wallet_id	UUID	NOT NULL REFERENCES wallets(id)	Source wallet (locked_balance)
expert_wallet_id	UUID	NOT NULL REFERENCES wallets(id)	Destination wallet (available_balance on release)
status	TEXT	NOT NULL DEFAULT 'HELD' CHECK (status IN ('HELD', 'RELEASED', 'FROZEN', 'REFUNDED', 'SPLIT'))	FROZEN set immediately when dispute is filed
held_at	TIMESTAMPZ	NOT NULL DEFAULT now()	
released_at	TIMESTAMPZ	NULL	

Table-level CHECK constraint (exactly one parent):

```
CONSTRAINT escrow_has_one_parent CHECK (
  (milestone_id IS NOT NULL AND engagement_id IS NULL) OR
  (milestone_id IS NULL AND engagement_id IS NOT NULL)
)
```

Two partial unique indexes (replacing old UNIQUE on milestone_id):

```
CREATE UNIQUE INDEX escrow_milestone_unique
ON escrow_accounts(milestone_id) WHERE milestone_id IS NOT NULL;

CREATE UNIQUE INDEX escrow_engagement_unique
ON escrow_accounts(engagement_id) WHERE engagement_id IS NOT NULL;
```

Index: status

Section 11 — Disputes (*NEW section*)

Table: `disputes`

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
engagement_id	UUID	NOT NULL REFERENCES engagements(id)
milestone_id	UUID	NOT NULL REFERENCES milestones(id)
dispute_id	TEXT	NOT NULL CHECK (dispute_id IN ('DISPUTE', 'DISPUTE_PENDING'))
dispute_at	TIMESTAMPZ	NOT NULL DEFAULT now()
released_at	TIMESTAMPZ	NULL
criteria_id	UUID	NOT NULL REFERENCES acceptance_criteria(id)

Column	Type	Constraints
escrow_account_id	UUID	NOT NULL REFERENCES escrow_accounts(id)
filed_by	UUID	NOT NULL REFERENCES users(id)
state	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (state IN ('PENDING', 'LAYER_1_EVAL', 'AUTO_RESOLVED', 'LAYER_2_MUTUAL', 'MUTUAL_RESOLVED', 'LAYER_3_FORCED', 'DISP
resolution_layer	INT	NULL CHECK (resolution_layer IN (1,2,3))
llm_confidence	FLOAT	NULL CHECK (llm_confidence BETWEEN 0 AND 1)
layer2_deadline	TIMESTAMP TZ	NULL
filed_at	TIMESTAMP TZ	NOT NULL DEFAULT now()
resolved_at	TIMESTAMP TZ	NULL
Index: engagement_id, milestone_id, state		

Table: dispute_resolution_reports

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
dispute_id	UUID	NOT NULL REFERENCES disputes(id)
reported_by	UUID	NOT NULL REFERENCES users(id)
reason_text	TEXT	NOT NULL
created_at	TIMESTAMP TZ	NOT NULL DEFAULT now()

Index: dispute_id

Section 12 — Add-On Phases & Messaging

Table: addon_phase_requests

Structurally changed — dual-approval timestamps replace single status approach.

Column	Type	Constraints
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()
engagement_id	UUID	NOT NULL REFERENCES engagements(id) ON DELETE CASCADE
submitted_by	UUID	NOT NULL REFERENCES users(id)

Column	Type	Constraints
trigger_sprint_status_id	UUID	NULL REFERENCES sprint_status_updates(id)
created_milestone_id	UUID	NULL REFERENCES milestones(id)
causal_chain	TEXT	NOT NULL
new_capability_requirements	TEXT	NOT NULL
why_invisible	TEXT	NOT NULL
status	TEXT	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'TECH_APPROVED', 'AWAITING_CEO', 'FULLY_APPROVED', 'REJECTED', 'MILESTONE_CREATED'))
tech_approved_at	TIMESTAMPTZ	NULL
tech_approved_by	UUID	NULL REFERENCES users(id)
ceo_approved_at	TIMESTAMPTZ	NULL
ceo_approved_by	UUID	NULL REFERENCES users(id)
rejected_at	TIMESTAMPTZ	NULL
rejected_by	UUID	NULL REFERENCES users(id)
rejection_reason	TEXT	NULL

UNIQUE constraint: (trigger_sprint_status_id) WHERE trigger_sprint_status_id IS NOT NULL — one SCOPE_EVOLUTION event triggers exactly one add-on.

```
CREATE UNIQUE INDEX addon_per_sprint_event
ON addon_phase_requests(trigger_sprint_status_id)
WHERE trigger_sprint_status_id IS NOT NULL;
```

Status transition logic in NestJS:

- PENDING → TECH_APPROVED: `tech_approved_at` set
- TECH_APPROVED → AWAITING_CEO: UI-facing label only
- AWAITING_CEO → FULLY_APPROVED: `ceo_approved_at` set → triggers milestone CREATE
- Any state → REJECTED: `rejected_at` set by either party
- FULLY_APPROVED → MILESTONE_CREATED: new milestone row written, `created_milestone_id` set

Index: `engagement_id, status`

Table: `sprint_comments` (NEW)

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
milestone_sprint_id	UUID	NOT NULL REFERENCES milestone_sprints(id) ON DELETE CASCADE	
commenter_id	UUID	NOT NULL REFERENCES users(id)	
comment_text	TEXT	NOT NULL	
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Route guard: write = TECH_TEAM only (`client_subtype = 'TECH_TEAM'`); read = TECH_TEAM + EXPERT. Separate from engagement-level `messages`.
Index: `milestone_sprint_id, created_at`

Table: `dod_item_comments` (*NEW*)

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
dod_item_id	UUID	NOT NULL REFERENCES milestone_dod_items(id) ON DELETE CASCADE	
commenter_id	UUID	NOT NULL REFERENCES users(id)	
comment_text	TEXT	NOT NULL	
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Route guard: write = TECH_TEAM only; read = TECH_TEAM + EXPERT; CEO hidden.
Index: `dod_item_id`

Table: `messages`

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
engagement_id	UUID	NOT NULL REFERENCES engagements(id) ON DELETE CASCADE	One channel per engagement; <code>engagement_id</code> is the thread key
sender_id	UUID	NOT NULL REFERENCES users(id)	Any of the three transactional roles; ADMIN read-only at route level
content	TEXT	NOT NULL	
attachments_json	JSONB	NULL	CHANGED from <code>attachment_url</code> TEXT — supports multiple files. Format: <code>[{url, filename, size_bytes}]</code> . NULL when no attachment.
timestamp	TIMESTAMPZ	NOT NULL DEFAULT now()	

Index: `engagement_id, timestamp`

Table: `message_reads` (*NEW*)

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
message_id	UUID	NOT NULL REFERENCES messages(id) ON DELETE CASCADE	
user_id	UUID	NOT NULL REFERENCES users(id)	
read_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Unique constraint: (`message_id, user_id`) — one read record per user per message

Unread count query:

```
SELECT COUNT(*) FROM messages m
WHERE m.engagement_id = :engagement_id
AND NOT EXISTS (
  SELECT 1 FROM message_reads r
  WHERE r.message_id = m.id AND r.user_id = :current_user_id
);
```

Index: `message_id, user_id` (unique implied)

Section 13 — Reviews & Outcome Signals

Table: `reviews`

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>	
<code>engagement_id</code>	UUID	NOT NULL REFERENCES <code>engagements(id)</code>	
<code>reviewer_id</code>	UUID	NOT NULL REFERENCES <code>users(id)</code>	
<code>target_id</code>	UUID	NOT NULL REFERENCES <code>users(id)</code>	
<code>rating</code>	INT	NOT NULL CHECK (<code>rating BETWEEN 1 AND 5</code>)	
<code>comment</code>	TEXT	NULL	
<code>structured_signals_json</code>	JSONB	NULL	TECH_TEAM structured signals; raw input for <code>outcome_signals</code>
<code>reviewer_role</code>	TEXT	NOT NULL CHECK (<code>reviewer_role IN ('CEO', 'TECH_TEAM', 'EXPERT')</code>)	

Unique constraint: (`engagement_id, reviewer_id`)

Index: `engagement_id, reviewer_id, target_id`

Table: `outcome_signals`

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>	
<code>engagement_id</code>	UUID	NOT NULL REFERENCES <code>engagements(id)</code>	
<code>reviewer_id</code>	UUID	NOT NULL REFERENCES <code>users(id)</code>	TECH_TEAM user
<code>review_id</code>	UUID	NOT NULL UNIQUE REFERENCES <code>reviews(id)</code>	NEW — FK back to the reviews row that generated this aggregate. UNIQUE enforces 1:1 between a review and its <code>outcome_signals</code> record.
<code>seam_signals_json</code>	JSONB	NOT NULL DEFAULT <code>'[]'</code>	Raw input kept for display layer. Format: <code>[{seam_code, signal_type, seam_role}]</code>
<code>ratings_json</code>	JSONB	NOT NULL DEFAULT <code>'{}'</code>	Per-dimension rating object

Unique constraint: (`engagement_id, reviewer_id`)

Index: `engagement_id`

Section 14 — Platform Decisions

Table: `platform_decisions`

Written by FastAPI only. Never written by NestJS. Never updated after insert.

Column	Type	Constraints
<code>id</code>	UUID	PK NOT NULL DEFAULT <code>gen_random_uuid()</code>
<code>decision_type</code>	TEXT	NOT NULL CHECK (<code>decision_type IN ('ELICITATION_SYNTHESIS', 'SPEC_AUTO_RETURN', 'SEAM_TIER_UPGRADE', 'PORTFOLIO_EVAL', 'SCENARIO_EVAL', 'DISP</code>)

Column	Type	Constraints
entity_type	TEXT	NOT NULL CHECK (entity_type IN ('project','capability_bid','acceptance_criterion','portfolio_submission'))
entity_id	TEXT	NOT NULL
llm_confidence	FLOAT	NULL CHECK (llm_confidence BETWEEN 0 AND 1)
decision	TEXT	NOT NULL
advisory_note	TEXT	NULL
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()
Index: decision_type, entity_type, entity_id, created_at		

Section 15 — Platform Settings (NEW)

Table: platform_settings

Singleton — exactly one row, seeded at deploy.

Column	Type	Constraints	Notes
id	UUID	PK NOT NULL DEFAULT gen_random_uuid()	
platform_wallet_id	UUID	NOT NULL UNIQUE REFERENCES wallets(id)	The seeded system wallet that receives PLATFORM_FEE ledger entries
platform_fee_pct	FLOAT	NOT NULL DEFAULT 0.05 CHECK (platform_fee_pct BETWEEN 0 AND 1)	5% — stored here so it can be changed without a code deploy
updated_at	TIMESTAMPZ	NOT NULL DEFAULT now()	

Seed SQL:

```
INSERT INTO platform_settings (platform_wallet_id, platform_fee_pct)
VALUES (:platform_wallet_id, 0.05);
```

Fee atomic ledger on milestone APPROVED:

```
-- Expert release (95%)
INSERT INTO wallet_transactions (wallet_id, amount, transaction_type, reference_id)
VALUES (:expert_wallet_id, milestone.payment_amount_vnd * (1 - ps.platform_fee_pct),
        'ESCROW_RELEASE', 'ESC_RELEASE:' || milestone.id);

-- Platform fee (5%)
INSERT INTO wallet_transactions (wallet_id, amount, transaction_type, reference_id)
VALUES (ps.platform_wallet_id, milestone.payment_amount_vnd * ps.platform_fee_pct,
        'PLATFORM_FEE', 'PLATFORM_FEE:' || milestone.id);
```

Draw / migrate in this exact sequence to avoid forward-reference violations:

1. users
2. client_profiles → users
3. expert_profiles → users
4. wallets → users
5. organizations → users
6. organization_members → organizations, users
7. user_subscriptions → users
8. withdrawal_requests → expert_profiles
9. wallet_transactions → wallets
10. elicitation_sessions → users
11. projects → users, elicitation_sessions
12. tech_team_profiles → users, projects
13. capability_footprints → projects
14. artifact_a → projects
15. artifact_b → projects
16. services → expert_profiles
17. engagements → projects, expert_profiles, services, organizations
18. virtual_accounts → (polymorphic entity_id – no FK; standalone)
19. platform_settings → wallets
20. expert_domain_depths → expert_profiles
21. expert_seam_claims → expert_profiles
22. expert_stack_tags → expert_profiles
23. portfolio_submissions → expert_profiles, expert_seam_claims
24. scenario_assessments → (standalone)
25. scenario_responses → scenario_assessments, expert_profiles, expert_seam_claims
26. spec_clarifications → artifact_a, expert_profiles, users
27. capability_bids → engagements
28. bid_revision_requests → capability_bids, users
29. bid_versions → capability_bids, bid_revision_requests, users
30. price_negotiations → capability_bids
31. bid_conflict_overrides → capability_bids, users
32. milestones → engagements
33. acceptance_criteria → milestones
34. revision_requests → acceptance_criteria, users
35. milestone_dod_items → milestones, acceptance_criteria
36. milestone_sprints → milestones, users
37. sprint_status_updates → milestones, milestone_sprints, users
38. milestone_submissions → milestones, expert_profiles
39. paygated_documents → milestones
40. escrow_accounts → milestones (nullable), engagements (nullable), wallets
41. disputes → engagements, milestones, acceptance_criteria, escrow_accounts, users
42. dispute_resolution_reports → disputes, users
43. addon_phase_requests → engagements, users, sprint_status_updates (nullable), milestones (nullable for created_milestone_id)
44. sprint_comments → milestone_sprints, users
45. dod_item_comments → milestone_dod_items, users
46. messages → engagements, users
47. message_reads → messages, users
48. reviews → engagements, users
49. outcome_signals → engagements, users, reviews
50. expert_seam_outcome_signals → engagements, expert_profiles, users, outcome_signals, expert_seam_claims
51. platform_decisions → (polymorphic entity_id – no FK; standalone)